TOP 30 ARRAY & STRING INTERVIEW CODING PROBLEMS

• EVERY JAVA DEVELOPER SHOULD SOLVE BEFORE INTERVIEW •

1. TWO SUM

Problem Statement :

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

Example : nums = [2,7,11,15], target = 9

Output : [0, 1] (Because 2 + 7 = 9)

APPROACH :

1. Traverse the array from left to right.
2. For each element, find its complement (target - nums[i]).
3. Check if complement exists in HashMap. If yes, return the stored index and current index.
4. Otherwise, store current element and its index in HashMap.

JAVA CODE :

```
import java.util.*;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
            map.put(nums[i], i);
        }
        return new int[0];
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(n)

In the worst case, we store all elements in map.

2. BEST TIME TO BUY AND SELL STOCK

Problem Statement :

You are given an array prices where prices[i] is the price of a given stock on the ith day. Find the maximum profit you can achieve.

Example : prices = [7,1,5,3,6,4]

Output : 5 (Buy at 1, Sell at 6)

APPROACH :

1. Keep track of the minimum price seen so far.
2. For each price, calculate profit = current_price - min_price.
3. Update max profit if current profit is greater.
4. Update min price if current price is smaller.

JAVA CODE :

```
class Solution {
    public int maxProfit(int[] prices) {
        if (prices == null || prices.length == 0) return 0;
        int minPrice = prices[0];
        int maxProfit = 0;
        for (int i = 1; i < prices.length; i++) {
            minPrice = Math.min(minPrice, prices[i]);
            int profit = prices[i] - minPrice;
            maxProfit = Math.max(maxProfit, profit);
        }
        return maxProfit;
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use a constant amount of extra space.

@aman_views

TOP 30 ARRAY & STRING INTERVIEW CODING PROBLEMS

Every Java Developer Should Solve Before Interview

3. MAXIMUM SUBARRAY (KADANE'S ALGORITHM)

Problem Statement:

Given an integer array nums, find the contiguous subarray with the largest sum, and return its sum.

Example:

nums = [-2,1,-3,4,-1,2,1,-5,4]

Output: 6 ([4,-1,2,1])

ALGORITHM (Kadane's Algorithm):

① Keep a running sum (currentSum) and best sum (maxSum).
② At each element, decide whether to extend previous subarray or start fresh.
③ currentSum = max(nums[i], currentSum + nums[i]).
④ Update maxSum = max(maxSum, currentSum).

JAVA CODE:

```
class Solution {
  public int maxSubArray(int[] nums) {
    int currentSum = nums[0];
    int maxSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
      currentSum = Math.max(nums[i], currentSum + nums[i]);
      maxSum = Math.max(maxSum, currentSum);
    }
   return maxSum;
  }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use a constant amount of extra space.

4. MOVE ZEROES

Problem Statement:

Given an array nums, move all 0's to the end while maintaining the relative order of non-zero elements.

Example:

nums = [0,1,0,3,12]

Output: [1,3,12,0,0]

ALGORITHM:

① Use a pointer (insertPos) to track position for next non-zero element.
② Traverse array; whenever a non-zero element is found, place it at insertPos and increment.
③ After traversal, fill remaining positions from insertPos to end with 0.

JAVA CODE:

```
class Solution {
  public void moveZeroes(int[] nums) {
    int insertPos = 0;
    for (int num : nums) {
      if (num != 0) {
        nums[insertPos++] = num;
      }
    }
    while (insertPos < nums.length) {
      nums[insertPos++] = 0;
   }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use a constant amount of extra space.

@aman_views

TOP 30 ARRAY & STRING INTERVIEW CODING PROBLEMS

Every Java Developer Should Solve Before Interview

5. MERGE INTERVALS

Problem Statement :

Given an array of intervals, merge all overlapping intervals.

Example :

intervals = [[1,3],[2,6],[8,10],[15,18]]

Output : [[1,6],[8,10],[15,18]]

ALGORITHM (Approach) :

1. Sort intervals by start time.
2. Initialize result list with the first interval.
3. For each subsequent interval, if it overlaps with the last merged interval (start <= last.end), merge by updating end.
4. Otherwise, add it as a new interval.

JAVA CODE :

```
import java.util.*;
class Solution {
  public int[][] merge(int[[] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List <int[]> result = new ArrayList<>();
    for (int[] interval : intervals) {
      if (result.isEmpty() ||
          result.get(result.size() - 1)[1] < interval[0]) {
          result.add(interval);
      } else {
         result.get(result.size() - 1)[1] =
          Math.max(result.get(result.size() - 1)[1], interval[1]);
      }
    }
    return result.toArray(new int[result.size()][]);
  }
}
```

Time Complexity : O(n log n)

Sorting dominates.

Space Complexity : O(n)

For result list.

6. PRODUCT OF ARRAY EXCEPT SELF

Problem Statement :

Given an array nums, return an array output such that output[i] is the product of all elements except nums[i], without using division.

Example :

nums = [1,2,3,4]

Output : [24,12,8,6]

ALGORITHM (Approach) :

1. Build a prefix product array where prefix[i] = product of all elements before i.
2. Build a suffix product array where suffix[i] = product of all elements after i.
3. output[i] = prefix[i] * suffix[i].
4. Optimize space by using output array itself for prefix, then multiply suffix on the fly.

JAVA CODE :

```
class Solution {
  public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] output = new int[n];
    output[0] = 1;
    for (int i = 1; i < n; i++) {
       output[i] = output[i - 1] * nums[i - 1];
    }
    int rightProduct = 1;
    for (int i = n - 1; i >= 0; i--) {
       output[i] *= rightProduct;
       rightProduct *= nums[i];
    }
    return output;
  }
}
```

Time Complexity : O(n)

Single pass from left to right and right to left.

Space Complexity : O(1)

Excluding output array.

@aman_views

TOP 30 ARRAY & STRING INTERVIEW CODING PROBLEMS

Every Java Developer Should Solve Before Interview

7. FIND DUPLICATE NUMBER

Problem Statement :

Given an array of n+1 integers where each integer is between 1 and n (inclusive), there is only one duplicate number. Find that duplicate number.

Example :

nums = [1,3,4,2,2]

Output : 2

ALGORITHM (Approach) :

1. Treat the array as a linked list where index points to nums[index].
2. Use slow and fast pointers to detect a cycle (duplicate number creates the cycle).
3. Once slow == fast, reset slow to nums[0].
4. Move both pointers one step at a time until they meet. That point is the duplicate number.

JAVA CODE :

```java
class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[0];
        int fast = nums[0];
        // Phase 1: Find the intersection point
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);
        // Phase 2: Find the entrance to the cycle
        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
}
```

Time Complexity : O(n)

We traverse the array at most twice.

Space Complexity : O(1)

We only use a constant amount of extra space.

8. MISSING NUMBER

Problem Statement :

Given an array containing n distinct numbers taken from 0 to n, find the missing number.

Example :

nums = [3,0,1]

Output : 2

ALGORITHM (Approach) :

1. Calculate the expected sum of numbers from 0 to n using the formula n*(n+1)/2.
2. Calculate the actual sum of array elements.
3. The missing number = expected sum - actual sum.
4. Return the missing number.

JAVA CODE :

```java
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : nums) {
            actualSum += num;
        }
        return expectedSum - actualSum;
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use a constant amount of extra space.

@aman_views

TOP 30 ARRAY & STRING INTERVIEW CODING PROBLEMS

Every Java Developer Should Solve Before Interview

9. ROTATE ARRAY

Problem Statement :

Given an array, rotate it to the right by k steps.

Example :

nums = [1,2,3,4,5,6,7], k = 3

Output : [5,6,7,1,2,3,4]

ALGORITHM (Approach) :

1. Normalize k using k = k % n (in case k > n).
2. Reverse the entire array.
3. Reverse first k elements.
4. Reverse remaining n-k elements.

JAVA CODE :

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k = k % n;
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }
    private void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}
```

Time Complexity : O(n)

We traverse the array a constant number of times.

Space Complexity : O(1)

We only use a constant amount of extra space.

10. CONTAINS DUPLICATE

Problem Statement :

Given an integer array, return true if any value appears at least twice.

Example :

nums = [1,2,3,1]

Output : true

ALGORITHM (Approach) :

1. Use a HashSet to track seen elements.
2. Traverse the array; if element already in set, return true.
3. Otherwise, add element to set.
4. If loop completes without duplicate, return false.

JAVA CODE :

```
import java.util.*;
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int num : nums) {
            if (!seen.add(num)) {
                return true;
            }
        }
        return false;
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(n)

In worst case, all elements are stored in the set.

11. MAJORITY ELEMENT (BOYER-MOORE VOTING)

Problem Statement :

Given an array of size n, find the majority element (appears more than n/2 times).

Example :

nums = [2,2,1,1,1,2,2]

Output : 2

ALGORITHM (Approach) :

① Initialize count = 0 and candidate = null.
② Traverse array; if count becomes 0, set current element as candidate.
③ If current element == candidate, increment count; else decrement count.
④ Candidate remaining at the end is the majority element.

JAVA CODE :

```
class Solution {
    public int majorityElement(int[] nums) {
        int count = 0;
        int candidate = 0;
        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }
            count += (num == candidate) ? 1 : -1;
        }
        return candidate;
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use a constant amount of extra space.

12. SORT COLORS (DUTCH NATIONAL FLAG)

Problem Statement :

Given an array with only 0s, 1s, and 2s, sort it in-place without using a library sort.

Example :

nums = [2,0,2,1,1,0,0]

Output : [0,0,1,1,2,2,2]

ALGORITHM (Approach) :

① Use three pointers: low, mid, high.
② If nums[mid] == 0, swap with nums[low], move both low and mid forward.
③ If nums[mid] == 1, just move mid forward.
④ If nums[mid] == 2, swap with nums[high], move high backward (mid stays).
⑤ Repeat until mid > high.

JAVA CODE :

```
class Solution {
    public void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;
        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums, low++, mid++);
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                swap(nums, mid, high--);
            }
        }
    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use constant extra space.

@aman_views

13. 3SUM

Problem Statement:

Given an integer array nums, return all the triplets [a, b, c] such that a + b + c == 0.

The result must not contain duplicate triplets.

Example:

nums = [-1, 0, 1, 2, -1, -4]

Output : [[-1, -1, 2], [-1, 0, 1]]

ALGORITHM (Approach):

① Sort the array.
② Fix one element (i) and use two pointers (left, right).
③ If sum < 0, move left pointer right.
④ If sum > 0, move right pointer left.
⑤ If sum == 0, add triplet and skip duplicates.
⑥ Repeat for all i.

JAVA CODE:

```java
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        Arrays.sort(nums);
        int n = nums.length;
        for (int i = 0; i < n - 2; i++) {
            if (i > 0 && nums[i] == nums[i-1]) continue;
            int left = i + 1, right = n - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    res.add(Arrays.asList(nums[i], nums[left], nums[right]));
                    while (left < right && nums[left] == nums[left+1]) left++;
                    while (left < right && nums[right] == nums[right-1]) right--;
                    left++; right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return res;
    }
```

Time Complexity : O(n2)
We use two loops.

Space Complexity : O(1)
We only use a constant amount of extra space.

14. TRAPPING RAIN WATER

Problem Statement:

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example:

height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]
Output : 6

ALGORITHM (Approach):

① Use two pointers: left and right.
② Track leftMax and rightMax.
③ Move the pointer with smaller height.
④ Water = max(0, leftMax - height[left]) or max(0, rightMax - height[right]).
⑤ Add water and move pointers.

JAVA CODE:

```java
class Solution {
    public int trap(int[] height) {
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (left < right) {
            if (height[left] < height[right]) {
                if (height[left] >= leftMax) leftMax = height[left];
                else water += leftMax - height[left];
                left++;
            } else {
                if (height[right] >= rightMax) rightMax = height[right];
                else water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }
```

Time Complexity : O(n)
We traverse the array once.

Space Complexity : O(1)
We only use a constant amount of extra space.

15. SEARCH IN ROTATED SORTED ARRAY

Problem Statement:

Given a rotated sorted array nums and a target value, return the index of target in the array. If not found, return -1.

Example:

nums = [4,5,6,7,0,1,2], target = 0

Output : 4

ALGORITHM (Approach):

1. Use binary search (left = 0, right = n-1).
2. Find mid = left + (right - left) / 2.
3. If nums[mid] == target, return mid.
4. Determine which half is sorted.
5. If target lies in the sorted half, search there, else search the other half.
6. Repeat until found or left > right.

JAVA CODE:

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            // Left half is sorted
            if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            } else {
                // Right half is sorted
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        return -1;
    }
}
```

Time Complexity : O(log n)
We use binary search.
Space Complexity : O(1)
We only use variables.

16. LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

Problem Statement:

Given a string s, find the length of the longest substring without repeating characters.

Example:

s = "abcabcbb"

Output : 3 ("abc")

ALGORITHM (Approach):

1. Use sliding window with two pointers.
2. Use a HashMap to store last index of each character.
3. If current character is seen and its index is within window, move left pointer.
4. Update max length at each step.
5. Move right pointer and continue.

JAVA CODE:

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);
            if (map.containsKey(ch) && map.get(ch) >= left)
                left = map.get(ch) + 1;
            map.put(ch), right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

Time Complexity : O(n)
We traverse the string once.
Space Complexity : O(min(n,m))
Where n = length of string, m = size of character set.

17. FIND DUPLICATE NUMBER

Problem Statement :

Given an array nums containing n + 1 integers where each integer is in the range [1, n] inclusive. There is only one repeated number in nums, return this duplicate number.

Example :

nums = [1,3,4,2,2]

Output : 2

ALGORITHM (Approach) :

1. Treat array as a linked list where next pointer is nums[i].
2. Use Floyd's Cycle Detection (Tortoise and Hare).
3. The meeting point is inside the cycle.
4. Move one pointer to start, keep the other at meeting point. Move both one step until they meet again. That node is the duplicate.

JAVA CODE :

```java
class Solution {
    public int findDuplicate(int[] nums) {
        // Phase 1 : Find intersection point
        int slow = nums[0], fast = nums[0];
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);
        // Phase 2 : Find entry point of cycle
        int ptr1 = nums[0], ptr2 = slow;
        while (ptr1 != ptr2) {
            ptr1 = nums[ptr1];
            ptr2 = nums[ptr2];
        }
        return ptr1;  // duplicate number
    }
}
```

Time Complexity : O(n)

We traverse the array a few times.

Space Complexity : O(1)

We use only constant extra space.

18. MISSING NUMBER

Problem Statement :

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

Example :

nums = [3,0,1]

Output : 2

ALGORITHM (Approach) :

1. The array contains numbers from 0 to n.
2. Calculate expected sum = n * (n + 1) / 2
3. Calculate actual sum of array.
4. Missing number = expected sum - actual sum.



PRO TIP: Use XOR for better performance and to avoid overflow in large inputs.

JAVA CODE :

```java
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : nums) {
            actualSum += num;
        }
        return expectedSum - actualSum;
    }
}
```

Time Complexity : O(n)

We traverse the array once.

Space Complexity : O(1)

We only use constant extra space.

@aman_views

19. ROTATE ARRAY

Problem Statement :

Given an array nums and an integer k, rotate the array to the right by k steps, where k is non-negative.

Example :

nums = [1,2,3,4,5,6,7], k = 3

Output : [5,6,7,1,2,3,4]

ALGORITHM (Approach) :

① Normalize k using k = k % n.

② Reverse the entire array.

③ Reverse the first k elements.

④ Reverse the remaining n - k elements.

JAVA CODE :

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k = k % n;
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }
    private void reverse(int[] nums, int l, int r) {
        while (l < r) {
            int temp = nums[l];
            nums[l] = nums[r];
            nums[r] = temp;
            l++;
            r--;
        }
    }
}
```

Time Complexity : O(n)

We traverse the array a few times.

Space Complexity : O(1)

We use only constant extra space.

20. PRODUCT OF ARRAY EXCEPT SELF

Problem Statement :

Given an integer array nums, return an array answer such that answer[i] is equal to the product of all the elements of nums except nums[i]. Do this without using division.

Example :

nums = [1,2,3,4]

Output : [24,12,8,6]

ALGORITHM (Approach) :

① Create answer array of size n.

② Traverse from left to right, store prefix product in answer array.

③ Traverse from right to left, multiply with suffix product.

JAVA CODE :

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        res[0] = 1;
        // Prefix product
        for (int i = 1; i < n; i++)
            res[i] = res[i - 1] * nums[i - 1];
        // Suffix product
        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] = res[i] * suffix;
            suffix *= nums[i];
        }
        return res;
    }
}
```

Time Complexity : O(n)

We traverse the array twice.

Space Complexity : O(n)

We use an extra array for result.

21. VALID ANAGRAM

Problem Statement :

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example :

s = "anagram", t = "nagaram"

Output : true

ALGORITHM (Approach) :

1. If lengths are not equal, return false.
2. Create a count array of size 26.
3. For each character in s, increment count.
4. For each character in t, decrement count.
5. If all counts are 0, return true, else false.

JAVA CODE :

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (char ch : s.toCharArray())
            count[ch - 'a']++;
        for (char ch : t.toCharArray())
            count[ch - 'a']--;
        for (int c : count) {
            if (c != 0) return false;
        }
        return true;
    }
}
```

Time Complexity : O(n)

We traverse both strings once.

Space Complexity : O(1)

We use a constant size array of 26.

22. VALID PALINDROME

Problem Statement :

Given a string s, return true if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Example :

s = "A man, a plan, a canal: Panama"

Output : true

ALGORITHM (Approach) :

1. Use two pointers i (start) and j (end).
2. Move i forward until an alphanumeric character is found.
3. Move j backward until an alphanumeric character is found.
4. Compare characters (case-insensitive). If not equal, return false.
5. If all characters match, return true.

JAVA CODE :

```
class Solution {
    public boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) {
            while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
            while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
            if (Character.toLowerCase(s.charAt(i))
                != Character.toLowerCase(s.charAt(j)))
                return false;
            i++;
            j--;
        }
        return true;
    }
}
```

Time Complexity : O(n)

We may traverse the string once.

Space Complexity : O(1)

We use only two pointers.

23. LONGEST SUBSTRING WITHOUT REPEATING CHARACTER

Problem Statement :

Given a string s, find the length of the longest substring without repeating characters.

Example :

s = "abcabcbb"

Output : 3

Explanation : The answer is "abc", with length 3.

ALGORITHM (Approach) :

① Use a sliding window with two pointers i and j.
② Use a HashMap to store the last index of each character.
③ Expand the window with j.
④ If character repeats, move i to one position ahead of the last occurrence.
⑤ Update max length at each step.

JAVA CODE :

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> map = new HashMap<>();
        int i = 0, maxLen = 0;
        for (int j = 0; j < s.length(); j++) {
            char ch = s.charAt(j);
            if (map.containsKey(ch) && map.get(ch) >= i) {
                i = map.get(ch) + 1;
            }
            map.put(ch, j);
            maxLen = Math.max(maxLen, j - i + 1);
        }
        return maxLen;
    }
}
```

Time Complexity : $O(n)$

We traverse the string once.

Space Complexity : $O(\min(n, m))$

Where m is the size of character set (e.g., 128 or 256).

24. LONGEST PALINDROME SUBSTRING

Problem Statement :

Given a string s, return the longest palindromic substring in s.

Example :

s = "babad"

Output : "bab"

Explanation : "bab" or "aba" are both valid answers.

ALGORITHM (Approach) :

① For each character, consider it as the center of a palindrome.
② Expand around the center for odd length palindrome.
③ Expand around the center and the next character for even length palindrome.
④ Keep track of the longest palindrome found.

JAVA CODE :

```
class Solution {
    public String longestPalindrome(String s) {
        if (s == null || s.length() < 2) return s;
        int start = 0, maxLen = 1;
        for (int i = 0; i < s.length(); i++) {
            int len1 = expandFromCenter(s, i, i);      // odd length
            int len2 = expandFromCenter(s, i, i + 1);  // even length
            int len = Math.max(len1, len2);
            if (len > maxLen) {
                start = i - (len - 1) / 2;
                maxLen = len;
            }
        }
        return s.substring(start, start + maxLen);
    }
    private int expandFromCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() &&
            s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1;
    }
}
```

Time Complexity : $O(n^2)$

We expand around each character.

Space Complexity : $O(1)$

We use only constant extra space.

25. GROUP ANAGRAMS

Problem Statement :

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Example :

strs = ["eat","tea","tan","ate","nat","bat"]

Output :

[["eat","tea","ate"],["tan","nat"],["bat"]]

ALGORITHM (Approach) :

① For each string, sort its characters.
② Use the sorted string as the key in a HashMap.
③ Store the original string in the list corresponding to that key.
④ Return all the lists from the map.

JAVA CODE :

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] ch = s.toCharArray();
            Arrays.sort(ch);
            String key = new String(ch);
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

Time Complexity : O(n * k log k)
Where n = number of strings, k = average length of string.

Space Complexity : O(n * k)
We store all strings in the map.

26. STRING TO INTEGER (ATOI)

Problem Statement :

Implement the myAtoi(string s) function which converts a string to a 32-bit signed integer (similar to C/C++ atoi function).
The function should handle leading spaces, optional sign, non-digit characters, and overflow.

Example :

s = "42"	Output : 42
s = " -42"	Output : -42
s = "4193 with words"	Output : 4193
s = "words and 987"	Output : 0
s = "-91283472332"	Output : -2147483648

ALGORITHM (Approach) :

① Skip leading spaces.
② Check for optional sign (+/-).
③ Read digits until a non-digit is found.
④ Convert the digits to number.
⑤ Handle overflow (clamp to 32-bit range).
⑥ Return the final number with sign.
⑦ If no digits found, return 0.

JAVA CODE :

```
class Solution {
    public int myAtoi(String s) {
        int i = 0, n = s.length();
        while (i < n && s.charAt(i) == ' ') i++;
        // Handle sign
        int sign = 1;
        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            if (s.charAt(i) == '-') sign = -1;
            i++;
        }
        // Convert digits
        long num = 0;
        while (i < n && Character.isDigit(s.charAt(i))) {
            int digit = s.charAt(i) - '0';
            num = num * 10 + digit;
            // Clamp to 32-bit range
            if (sign == 1 && num > Integer.MAX_VALUE) return Integer.MAX_VALUE;
            if (sign == -1 && -num < Integer.MIN_VALUE) return Integer.MIN_VALUE;
            i++;
        }
        return (int)(sign * num);
    }
}
```

Time Complexity : O(n)
We traverse the string once.

Space Complexity : O(1)
We use constant extra space.

27. REVERSE WORDS IN A STRING

Problem Statement :

Given a string s, reverse the order of words in the string.
Words are separated by a single space.
Return a string with words reversed in order without leading or trailing spaces.

Example :

s = "the sky is blue"

Output : "blue is sky the"

ALGORITHM (Approach) :

① Trim leading and trailing spaces.
② Split the string by spaces into words.
③ Reverse the list of words.
④ Join the words with a single space.
⑤ Return the resulting string.

JAVA CODE :

```java
class Solution {
    public String reverseWords(String s) {
        s = s.trim();
        String[] words = s.split("\\s+");
        int i = 0, j = words.length - 1;
        while (i < j) {
            String temp = words[i];
            words[i] = words[j];
            words[j] = temp;
            i++;
            j--;
        }
        return String.join(" ", words);
    }
}
```

Time Complexity : O(n)

We traverse the string and reverse the words.

Space Complexity : O(n)

We use extra space for storing words.

28. MINIMUM WINDOW SUBSTRING

Problem Statement :

Given two strings s and t, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such window, return "".

Example :

s = "ADOBECODEBANC"
t = "ABC"
Output : "BANC"

ALGORITHM (Approach) :

① Use a HashMap to store frequency of characters of t.
② Use two pointers (left, right) to form a window.
③ Expand the window by moving right pointer and update the frequency map.
④ When all characters are matched, try to shrink the window from left to get minimum length.
⑤ Update the result whenever a valid smaller window is found.

JAVA CODE :

```java
class Solution {
    public String minWindow(String s, String t) {
        if (s.length() < t.length()) return "";
        Map<Character, Integer> map = new HashMap<>();
        for (char c : t.toCharArray())
            map.put(c, map.getOrDefault(c, 0) + 1);
        int required = map.size(), formed = 0;
        int left = 0, right = 0, minLen = Integer.MAX_VALUE;
        int start = 0;
        while (right < s.length()) {
            char c = s.charAt(right);
            map.put(c, map.getOrDefault(c, 0) - 1);
            if (map.get(c) == 0) formed++;
            while (left <= right && formed == required) {
                if (right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    start = left;
                }
                char lc = s.charAt(left);
                map.put(lc, map.get(lc) + 1);
                if (map.get(lc) > 0) formed--;
                left++;
            }
            right++;
        }
        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
    }
}
```

Time Complexity : O(n)

We traverse the string using the sliding window.

Space Complexity : O(k)

Where k is the number of unique characters in t.

29. LONGEST COMMON PREFIX

Problem Statement :

Write a function to find the longest common prefix string amongst an array of strings.

Example :

strs = ["flower","flow","flight"]

Output :

"fl"

strs = ["dog","racecar","car"]

Output : ""

ALGORITHM (Approach) :

① If array is empty, return "".
② Take the first string as prefix.
③ Compare it with each other string character by character.
④ Reduce the prefix length when mismatch occurs.
⑤ Return the final prefix.

JAVA CODE :

```java
class Solution {
    public String longestCommonPrefix(String[] strs) {
        if (strs == null || strs.length == 0) return "";
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}
```

Time Complexity : O(n * m)

Where n = number of strings,
m = length of shortest string.

Space Complexity : O(1)

We use only constant extra space.

30. VALID PARENTHESES

Problem Statement :

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

Example :

s = "()[]{}" Output : true
s = "(" Output : false
s = "(]){}" Output : true

ALGORITHM (Approach) :

① If length of string is odd, return false.
② Use a stack to store opening brackets.
③ For each character :
- If it's an opening bracket, push to stack.
- If it's a closing bracket, check if stack is empty or top doesn't match.
- If not matched, return false.

④ At the end, if stack is empty, return true; otherwise false.

JAVA CODE :

```java
class Solution {
    public boolean isValid(String s) {
        if (s.length() % 2 != 0) return false;
        Stack<Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if (c == ')' && top != '(' ||
                    (c == '}' && top != '{' ||
                    (c == ']' && top != '[')) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
}
```

Time Complexity : O(n)

We traverse the string once.

Space Complexity : O(n)

In worst case, all opening brackets are stored in stack.